

Electricity Load and Renewable Forecasting: A Comparative Analysis of LSTM, GRU, and TCN Models

Group 11

AISE4010: Deep Learning for Time Series Data

Western University

London, Ontario

Cole Book – 251279414

Adam Dunn – 251296728

Jaemin Cho – 251303645

Bryce Nielsen – 251296057

Abstract—As power systems integrate increasing amounts of wind and solar generation, short-term load forecasting is critical for maintaining grid stability and minimizing balancing costs. This project forecasts hourly electricity load in Germany using three deep learning architectures: Long Short-Term Memory (LSTM) networks, Gated Recurrent Units (GRUs), and Temporal Convolutional Networks (TCNs). We build a multivariate time series that combines historical load with onshore wind, solar generation, and calendar features (hour, day of week, month). After standardized preprocessing, 24-hour sliding-window sequence generation, and chronological train-validation-test splitting, we tune and evaluate each model using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and training time. The multivariate GRU achieves the best accuracy with MAE 463 MW and RMSE 601 MW, while a tuned TCN attains MAE 515 MW but trains about 3.5× faster. A univariate GRU trained only on load history yields substantially higher error (MAE 555 MW), confirming the value of renewable and calendar features. Overall, GRUs are preferred when minimizing grid imbalance is the primary objective, whereas TCNs offer a strong accuracy–efficiency trade-off for time-critical or resource-constrained deployments.

I. Introduction

Modern power systems must continuously balance supply and demand. In Germany’s “Energiewende,” fast-growing onshore wind and solar generation introduce additional variability and uncertainty, making accurate short-term load forecasting (STLF) a core operational tool. Reliable forecasts support unit commitment, economic dispatch, reserve scheduling, and market participation, while reducing costly balancing actions and curtailment.

Classical statistical models such as ARIMA or exponential smoothing rely on linear assumptions and have difficulty representing nonlinear interactions among load, renewable output, and calendar effects. Deep learning models can instead learn complex temporal patterns directly from data. Recurrent architectures like LSTMs and GRUs maintain internal states to model long-range dependencies, whereas Temporal Convolutional Networks (TCNs) use dilated causal convolutions to obtain large receptive fields without sequential computation.

In this work we forecast next-hour German load using the previous 24 hours of system conditions. Our contributions are threefold: (1) we design a transparent preprocessing pipeline

that converts raw operational data into model-ready sequences; (2) we compare LSTM, GRU, and TCN architectures under a common experimental setup, analyzing both prediction accuracy and computational cost; and (3) we quantify the impact of renewable and calendar features by contrasting a multivariate GRU with a univariate GRU baseline. The analysis is grounded in real operational data and standard error metrics, yielding practical guidance for model selection in renewable-rich grids.

II. Methodology

A. Dataset Description

We use the Open Power System Data (OPSD) Time Series package [1], which provides cleaned hourly electricity data for European countries. From this dataset we extract German variables most relevant to load and renewable integration. The target is actual load reported to ENTSO-E (DE_load_actual_entsoe_transparency, MW). Explanatory physical variables are onshore wind generation (DE_wind_onshore_generation_actual) and solar generation (DE_solar_generation_actual).

After loading the data, we retain these columns, convert timestamps to Python datetime objects, and set them as the DataFrame index to enforce chronological ordering. We also engineer simple calendar features, hour of day, day of week (0–6), and month (1–12), so that the models can exploit daily usage cycles, weekday–weekend differences, and seasonal trends.

To assess feature relationships and justify a multivariate approach, we compute the Pearson correlation matrix shown in Fig. 1. Load is moderately correlated with solar generation and weakly with hour of day, while day of week shows a negative correlation, reflecting lower weekend demand. Wind and solar exhibit low linear correlation with load, but their nonlinear influence can still be captured by deep models.

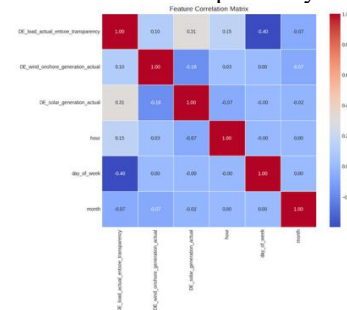


Figure 1: Feature correlation matrix for Load, Wind Generation, Solar Generation, and calendar features (hour, day of week, month).

We also perform seasonal decomposition of the load series using a 24-hour period (Fig. 2). The decomposition reveals a strong diurnal component, slower seasonal trends, and residual noise. This clear multi-scale temporal structure motivates the use of recurrent and convolutional architectures capable of learning both short- and long-term dependencies.

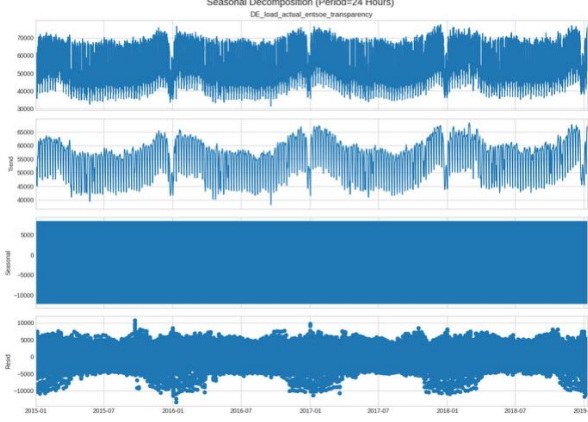


Figure 2: Seasonal decomposition of the German load time series (period = 24 hours), showing the observed signal, trend component, daily seasonal pattern, and residual fluctuations

B. Data Cleaning and Preprocessing

Operational data contain occasional missing entries. After converting columns to numeric types, we count missing values and handle them via time-based interpolation on the indexed series, followed by forward/backward filling for any remaining gaps. This preserves temporal continuity better than simple mean imputation.

We check for extreme outliers in the load series using Z-scores; the fraction exceeding an absolute Z-score of three is very small, indicating a reasonably well-behaved distribution. To stabilize optimization and place features on comparable scales, we standardize all variables with scikit-learn’s StandardScaler, fitting the scaler only on the training portion to avoid leakage, then applying it to validation and test sets.

The standardized data are split chronologically into 70% training, 15% validation, and 15% test. This mimics real-world forecasting, where models are trained on the past and evaluated on unseen future data.

C. Sequence Generation

Keras models expect inputs of shape (samples, time_steps, features). We therefore use a sliding-window procedure to create fixed-length sequences. A helper function create_sliding_windows constructs, for each index i , a 24-hour context window of all features as the input and the load value 24 hours ahead as the target. This yields tensors X_{train} , X_{val} , X_{test} with shape $(N, 24, N_{features})$ and corresponding label vectors y_{train} , y_{val} , y_{test} .

For multivariate models, the feature dimension includes load, wind, solar, and all calendar features. For the univariate GRU baseline, we build separate sequences containing only standardized load, keeping the same 24-hour look-back to maintain a fair comparison.

D. Model Architectures and Training Procedures

1. LSTM Baseline and Lightweight Hyperparameter Search:

The first architecture is a stacked LSTM network used as a baseline recurrent model. It consists of two LSTM layers (the first returning the full hidden-state sequence, the second producing a final summary state), followed by dropout, a dense ReLU-activated hidden layer, and a linear output neuron predicting next-hour load. The model is trained with the Adam optimizer (learning rate $1e-3$) and mean squared error (MSE) loss.

We run a lightweight grid search over LSTM units {32, 64, 128} and learning rates { $1e-3$, $5e-4$ }, keeping batch size 64. Each configuration is trained for up to 50 epochs with early stopping on validation loss, and the best configuration by validation RMSE is retrained on the full training set. The optimized LSTM predictions on the test set are shown in Fig. 3. The model captures the overall diurnal pattern but underestimates peaks, resulting in a relatively high MAE of 771.94 MW.

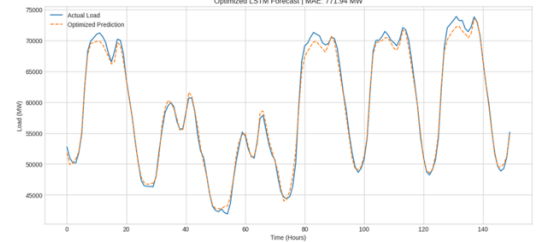


Figure 3: Optimized LSTM forecast versus actual load over 150 test hours, showing accurate daily patterns but consistent underestimation of peaks (MAE=771.94MW)

2. GRU with Multivariate vs. Univariate Inputs:

The second architecture replaces LSTM layers with GRU cells. The helper function build_gru_model constructs two stacked GRU layers (sequence-returning then summary), followed by dropout and a dense output layer. Training again uses Adam with learning rate $1e-3$ and MSE loss.

Before final training we tune the number of GRU units. Configurations with 32, 64, and 128 units are evaluated on the multivariate inputs, each trained for five epochs with early stopping to obtain a quick estimate of validation MAE. As shown in Fig. 4, the 64-unit GRU performs best and is selected for subsequent experiments.

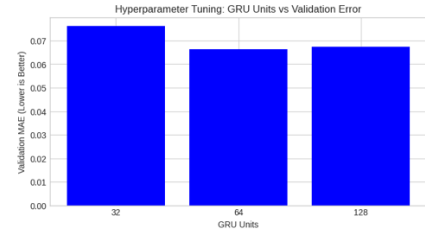


Figure 4: Validation MAE for GRU models with 32, 64, and 128 units. The 64-unit configuration achieves the lowest error and is selected for final training.

Using this configuration, we train two GRU models. The multivariate GRU uses all standardized features (load, wind, solar, hour, day of week, month), whereas the univariate GRU is trained only on load history. To reduce overfitting on the simpler univariate input, the multivariate GRU is trained for up to 50 epochs with early stopping and the univariate GRU for up to 30 epochs. Fig. 5 compares their predictions on the test set: the multivariate GRU closely follows load fluctuations, particularly peaks and troughs, while the univariate model exhibits noticeably larger errors.

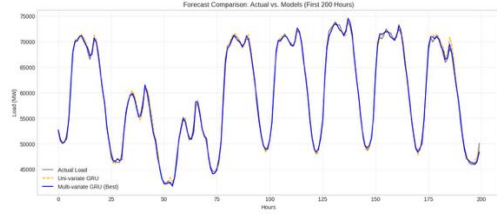


Figure 5: GRU forecast comparison for the first 200 test hours. The multivariate GRU tracks load patterns more accurately than the univariate GRU, demonstrating the value of renewable and calendar features.

3. Temporal Convolutional Network (TCN) with Bayesian Hyperparameter Optimization:

The final architecture is a TCN that replaces recurrence with stacks of 1D causal dilated convolutions. Each residual block applies two dilated convolutions with exponentially increasing dilation, batch normalization, ReLU activations, and spatial dropout. A skip connection around each block preserves gradient flow. Stacking several such blocks yields a receptive field that spans the full 24-hour context while remaining parallelizable.

To explore architectural choices efficiently, we define a Keras Tuner hypermodel `build_tcn_hypermodel` that exposes the number of residual blocks (2–5), number of filters (32, 64, 96, 128), kernel size (3 or 5), and spatial dropout rate (0.1–0.3) as hyperparameters. The model is compiled with Adam and MSE loss. Bayesian Optimization runs up to 40 trials, monitoring validation loss and using early stopping to discard poor configurations. The best-performing TCN is retrained for up to 50 epochs with batch size 64.

The optimized TCN forecast is shown in Fig. 6. It tracks rapid ramps and daily structure more accurately than the LSTM and approaches the GRU’s performance. Training and validation loss curves in Fig. 7 demonstrate rapid, stable convergence, highlighting the efficiency of the convolutional approach.

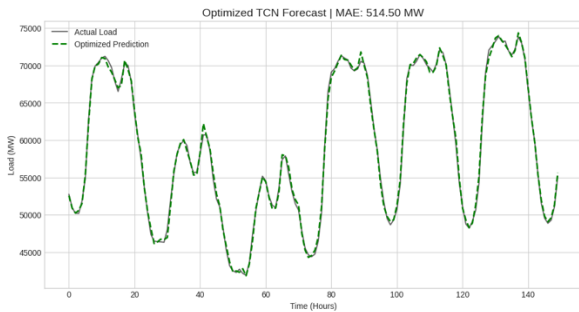


Figure 6: Optimized TCN forecast versus actual load for the first 150 test hours. The TCN captures rapid load transitions and achieves competitive accuracy with substantially lower training time than recurrent models.

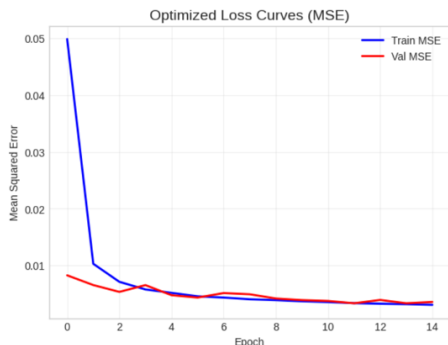


Figure 7: Training and validation MSE for the optimized TCN model, showing fast convergence and stable validation performance during training.

III. Results and Analysis

All three optimized models are evaluated on the same held-out test set. Predictions and targets are inverse-transformed back to megawatts so that MAE and RMSE are reported in physical units.

A. Overall Performance Comparison

Table 1 summarizes the test performance of the tuned LSTM, multivariate GRU, and best TCN in terms of MAE, RMSE, and total training time:

Model	MAE (MW)	RMSE (MW)	Training Time (s)
LSTM	771.94	955.39	39.01
GRU	463.21	600.71	220.99
TCN	514.50	666.71	63.99

Table 1: Comparison of MAE, RMSE, and training time across the three models.

A visual comparison of these metrics is given in Fig. 8. The multivariate GRU clearly yields the lowest error, reducing MAE by more than 300 MW relative to the LSTM and by roughly 50 MW relative to the TCN. Its lower RMSE indicates better handling of large deviations, which is important when large under- or over-predictions can trigger costly balancing actions. The TCN, while slightly less accurate, substantially outperforms the LSTM and trains about 3.5× faster than the GRU, making it attractive when models must be retrained frequently.



Figure 8: Comparison of MAE, RMSE, and training time for LSTM, GRU, and TCN.

B. Analysis of Accuracy

A key part of our analysis is assessing how much renewable generation and calendar features contribute to predictive performance. Using the same tuned architecture (64 units), we compare a multivariate GRU to a univariate GRU trained only on historical load:

- Univariate GRU: MAE 554.80 MW, RMSE 719.02 MW.
- Multivariate GRU: MAE 463.21 MW, RMSE 600.71 MW.

The inclusion of wind, solar, and calendar features reduces MAE by approximately 91.6 MW, corresponding to an improvement of about 16.5% relative to the univariate baseline. RMSE drops by over 118 MW. This confirms that renewable generation data and simple time-of-day and day-of-week indicators capture important variation in the net load that cannot be inferred from load history alone.

Visual inspection of the actual-versus-predicted plot for the multivariate GRU shows that the model closely follows daily peaks and troughs, including weekend demand reductions and midday solar-driven effects. The residuals histogram is centered close to zero with a roughly symmetric shape, indicating limited systematic bias and a concentration of errors within a moderate range of magnitudes.

C. Analysis of Efficiency

For the tuned LSTM and GRU models, we examine training and validation loss and MAE curves over epochs. The baseline LSTM shows slower convergence and a wider gap between training and validation curves, suggesting potential

underfitting under the chosen architecture and hyperparameters, despite early stopping. In contrast, the multivariate GRU displays smoother convergence and a smaller validation–training gap, supporting the interpretation that its architecture better matches the complexity of the dataset.

The Keras Tuner runs for the TCN reveal that deeper networks with 3–4 residual blocks and moderate filter widths tend to achieve the best validation performance, confirming the benefit of a sufficiently large receptive field over the 24-hour history. The optimized TCN’s loss curves also stabilize quickly due to the use of early stopping and dropout, which helps prevent overfitting despite the model’s capacity.

D. Efficiency Considerations

While accuracy is crucial for minimizing balancing costs, retraining speed and computational efficiency matter when models must be updated regularly with new data. The LSTM baseline is relatively fast to train but provides the worst accuracy, making it less attractive despite its simplicity. The GRU offers the best accuracy but at the highest computational cost due to longer training and hyperparameter tuning. The TCN strikes a middle ground: it reaches accuracy close to the GRU while training significantly faster thanks to its fully convolutional, parallelizable architecture and the use of Bayesian hyperparameter optimization to avoid extensive manual search.

IV. Discussion

The experiments highlight several important insights for load forecasting in renewable-rich power systems.

First, incorporating renewable generation and calendar features dramatically improves predictive performance. The comparison between multivariate and univariate GRUs shows that wind and solar data effectively explain part of the variability in net demand that would otherwise appear as noise. In operational terms, this aligns with the concept of residual load: the demand that must be met by conventional generators after accounting for renewable output.

Second, among the deep learning architectures studied, the GRU is particularly well suited to this dataset. Its simplified gating structure appears sufficient to model the dominant temporal patterns in German load without the added complexity of LSTM cells. The significant performance gap between the GRU and the LSTM under similar training conditions suggests that, for medium-sized datasets with relatively short look-back windows (24 hours), GRUs can often outperform LSTMs while requiring fewer parameters.

Third, the TCN results demonstrate that convolutional architectures are a competitive alternative to recurrent networks for time series forecasting. The TCN not only achieves strong accuracy but also trains substantially faster because convolutions over time can be computed in parallel. This property is highly attractive for real-time environments where models may need to be retrained daily or even hourly with new data.

There are, however, several limitations. Our models are trained using a fixed 24-hour look-back window, which may not fully capture longer-term weekly or seasonal

dependencies. We also rely only on aggregated national-level data; regional or nodal measurements could exhibit different patterns. Moreover, we do not explicitly incorporate exogenous weather variables such as temperature or wind speed, which are known to be informative for both demand and renewable generation. Finally, the hyperparameter searches for LSTM and GRU are deliberately lightweight to manage runtime in the notebook environment; a more exhaustive search could further improve their performance.

V. Conclusion

This project developed a complete deep learning pipeline for short-term electricity load forecasting in Germany using real operational data from the Open Power System Data platform. After constructing a multivariate time series that combines load, onshore wind, solar generation, and calendar features, we standardized the data, generated sliding-window sequences, and trained three classes of models: an LSTM baseline, a tuned GRU, and a TCN optimized via Bayesian hyperparameter search.

The multivariate GRU emerged as the most accurate model, achieving an MAE of approximately 463 MW and an RMSE of 601 MW on the test set. The TCN delivered slightly lower accuracy but significantly shorter training times, offering a compelling balance between precision and efficiency. The univariate GRU baseline highlighted the importance of renewable generation and calendar features, with multivariate inputs yielding more than a 16% reduction in MAE.

For system operators and planners, these results suggest that GRU-based architectures are a strong choice when minimizing forecast error is paramount, while TCNs are attractive in settings where frequent retraining or deployment on constrained hardware is required. Future work could extend this study in several directions: incorporating detailed weather forecasts as additional exogenous inputs, exploring hybrid models that combine convolutional and recurrent layers (e.g., CNN-LSTM or TCN-GRU hybrids), experimenting with longer look-back windows to capture weekly and seasonal structure, and evaluating probabilistic forecasting methods that output predictive distributions rather than point forecasts. These extensions would further enhance the robustness and practical value of deep learning approaches for managing renewable-rich power systems.

Project Code

The complete code is available at the following GitHub repository:

https://github.com/coleb9/Time-Series-Project/blob/main/Group_11_Time_Series_Project_Final.ipynb

References

- [1] Open Power System Data, "Data Package Time Series," 2020. [Online]. Available: https://data.open-power-system-data.org/time_series/.
- [2] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [3] K. Cho et al., "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation," *arXiv preprint arXiv:1406.1078*, 2014. [4] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," *arXiv preprint arXiv:1803.01271*, 2018.